Rapport de stage

Développement d'un site internet permettant la location de véhicule

Centre de la Gabrielle, 6 rue de la Gabrielle, Claye-Souilly, 77410
IUT de Villetaneuse – Université Sorbonne Paris Nord,
99 Av. Jean Baptiste Clément, 93430 Villetaneuse

Rédigé par Denis-Can YUKER, Étudiant en deuxième année d'un BUT Informatique
Maître de Stage : Flavien TCHAPGA
Tuteur de Stage : Jean-Luc ANGOT
Dates du stage : Du 22 janvier 2024 au 18 mars 2024

Remerciements

Je remercie Mme **Ozlem Saylam** pour son initiative d'avoir parlé de moi à son entreprise. Je devais trouver rapidement un stage car nous approchions de la fin de la période initiale de recherche de stage.

Je souhaite ensuite remercier M. **Jean-Luc Angot**, qui m'a accepté dans son entreprise. Je le remercie pour son accueil, son soutien et sa guidance tout au long de ce stage.

Je remercie également Mme **Haïfa Zargayouna** d'avoir accepté mon BOS et permis la réalisation de ce stage. Son approbation a été importante pour faire ce stage.

Je voudrais aussi remercier mon tuteur universitaire, M. **Flavien Tchapga**, pour son accompagnement, ses conseils et son soutien durant mon stage.

Table des matières

Introduction 4

I. Cadre du stage 5

I.1. Présentation de l'entreprise 5

I.1.1. Service pour les personnes handicapés 5

I.1.2. Structure organisationnelle 5

I.1.3. Équipe Informatique et Support Technique 6

I.2. Objectifs du stage au Centre de la Gabrielle 7

I.2.1. Présentation du problématique 7

I.2.2. Objectifs de l'entreprise 8

II. Missions et projets réalisés 8

II.1. Ma mission accordée au stage 8

II.2. Approche de Développement et Choix Technologiques 8

II.2.1. Consignes et méthodologie de travail 8

II.2.2. Outils de conception et de développement (VSCode, Figma) 10

II.2.3. Organisation du projet et architecture MVC 10

II.2.4. Programmation Front-End et maquette 11

II.2.5. Création de la base de données avec programmation Back-End. 16

II.2.6. Résultats du site internet 20

III. Ma première expérience professionnelle 20

III.1. Challenges rencontrés 20

III.2. Mon avis sur cette expérience 21

Conclusion 21

Glossaire 22

Annexes 23

Introduction

Le stage que j'ai effectué était un stage de ma deuxième année de BUT Informatique. Après avoir envoyé mon CV et ma lettre de motivation à un contact, j'ai été accepté par l'entreprise Centre de la Gabrielle. Cette entreprise est spécialisée dans l'accompagnement des personnes en situation de handicap mental et est située à Claye-Souilly.

Le poste qui m'avait été proposé dans cette entreprise était celui de développeur full-stack. J'étais très intéressé par ce poste car elle est en lien direct avec mes études actuelles. Je me suis dit pourquoi pas essayer de faire ce stage, je pourrais découvrir le « monde du travail » ainsi que le métier du développeur full-stack. Ma mission principale lors de ce stage était de concevoir un site internet intranet permettant la réservation de véhicules.

Dans mon rapport, je vais d'abord présenter le cadre du stage en décrivant l'entreprise. Ensuite, j'aborderai les objectifs du stage, en expliquant la problématique à laquelle j'ai été confronté ainsi que les objectifs fixés par l'entreprise. Après avoir établi ce contexte, je détaillerai les missions et les projets que j'ai réalisés. Par la suite, j'analyserai les résultats obtenus. Enfin, je conclurai en résumant les enseignements que j'ai tirés de cette expérience durant le stage.

I. Cadre du stage

I.1. Présentation de l'entreprise

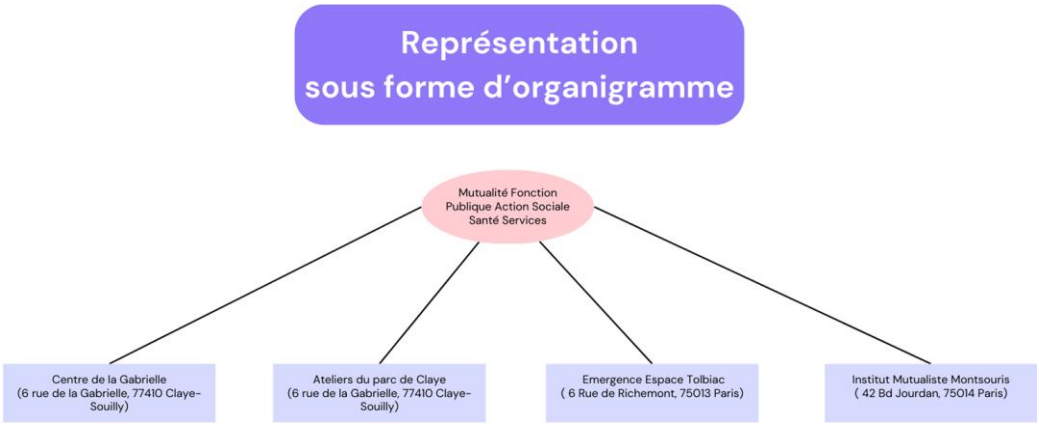
I.1.1. Service pour les personnes handicapés

Le Centre de la Gabrielle est une entreprise spécialisée dans l'accompagnement des personnes en situation de handicap située à Claye-Souilly depuis 1972.

Ce centre met en place des projets personnalisés d'accompagnement avec les personnes accueillies et leurs familles comme le socio-éducatif, la thérapeutique, la nutrition ainsi que les troubles sévères du comportement alimentaire et le numérique.

I.1.2. Structure organisationnelle

Le Centre de la Gabrielle est affilié à la MFPASS (Mutualité Fonction Publique Action Sociale Santé Services). La MFPASS est une mutuelle qui regroupe différentes structures. Elle est affilée à plusieurs structure telles que les Ateliers du Parc de Claye et Emergence Espace Tolbiac.



Le Centre de la Gabrielle est constitué de plus de 300 professionnels accompagnant 500 personnes en situation de handicap mental et est en partenariat avec plus de 30 entreprises. La directrice générale du Centre de la Gabrielle est Bernadette Grosyeux.

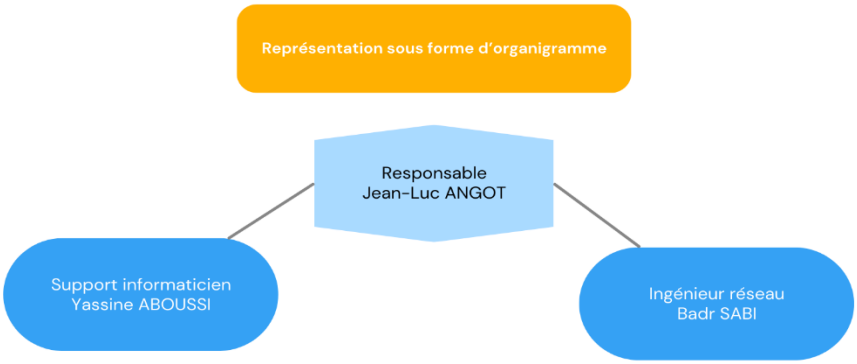
I.1.3. Équipe Informatique et Support Technique

Le Centre de la Gabrielle contient un secteur informatique composé de 2 employés et d'un responsable. Parmi ces employés, il y a un support technicien et un ingénieur système et réseau.

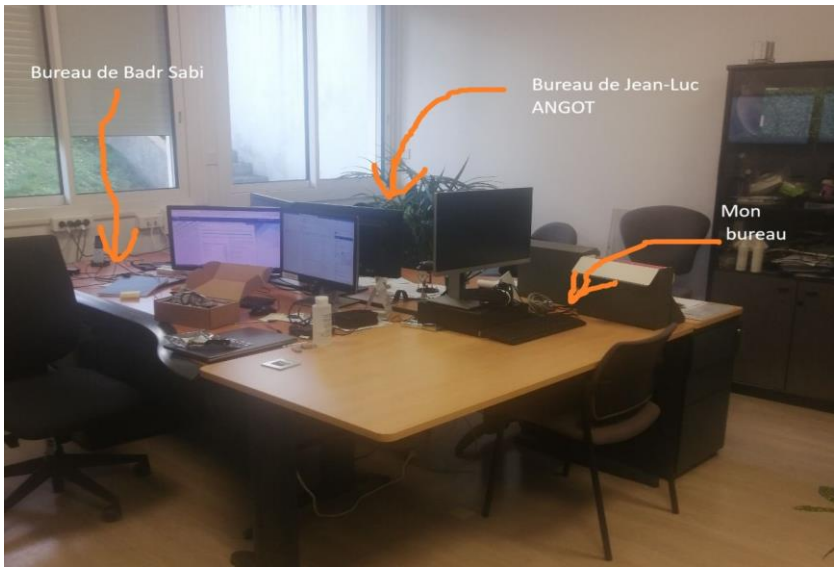
Le responsable est Jean-Luc ANGOT (qui est aussi mon encadrant), il encadre les 2 personnes et gère le service informatique au niveau des projets, de la sécurité et de la GRPD. Il est capable de remplacer les 2 autres personnes et participe au développement de l'entreprise.

Le premier employé est Badr SABI, qui est un ingénieur réseau.

Le dernier employé est Yassine ABOUSSI, qui est un support informaticien.



Il y a 2 salles dans le secteur informatique:

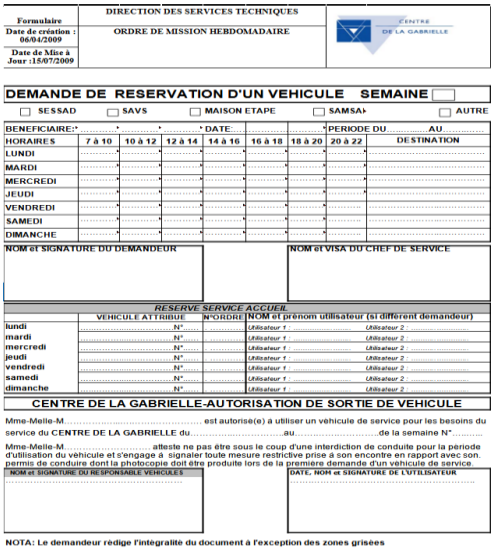


Salle principale

I.2. Objectifs du stage au Centre de la Gabrielle

I.2.1. Présentation du problématique

Dans le service informatique, les employés prennent en charge les demandes de véhicules. Les encadrants doivent faire une demande de véhicule pour pouvoir les utiliser. Auparavant, ces demandes étaient gérées sur papier et des fois, les employés étaient occupés ou qu'il y avait d'autres demandes en attente. Les encadrants devaient alors remplir des formulaires physiques pour réserver un véhicule. Le temps de transmettre entraînait des retards. Actuellement, le service informatique se retrouve donc avec une tonne de papier à traiter. Il existe 2 types de formulaires pour réserver un véhicule: ponctuelle et hebdomadaire. Voici les formulaires en papier qui étaient utilisés:



Formulaire | DIRECTION DES SERVICES TECHNIQUES
Date de création : 06/04/2009 | ORDRE DE MISSION HEBDOMADAIRE
Date de Mise à Jour :15/07/2009

DEMANDE DE RESERVATION D'UN VEHICULE SEMAINE
☐ SESSAD ☐ SAVS ☐ MAISON ETAPE ☐ SAMSAH ☐ AUTRE

BENEFICIAIRE:* DATE: PERIODE DU AU

HORAIRES	7 à 10	10 à 12	12 à 14	14 à 16	16 à 18	18 à 20	20 à 22	DESTINATION
LUNDI								
MARDI								
MERCREDI								
JEUDI								
VENDREDI								
SAMEDI								
DIMANCHE								

NOM et SIGNATURE DU DEMANDEUR | NOM et VISA DU CHEF DE SERVICE

RESERVE SERVICE ACCUEIL

	VEHICULE ATTRIBUE	N°ORDRE	NOM et prénom utilisateur (si différent demandeur)
lundi	N°		Utilisateur 1 : Utilisateur 2 :
mardi	N°		Utilisateur 1 : Utilisateur 2 :
mercredi	N°		Utilisateur 1 : Utilisateur 2 :
jeudi	N°		Utilisateur 1 : Utilisateur 2 :
vendredi	N°		Utilisateur 1 : Utilisateur 2 :
samedi	N°		Utilisateur 1 : Utilisateur 2 :
dimanche	N°		Utilisateur 1 : Utilisateur 2 :

CENTRE DE LA GABRIELLE-AUTORISATION DE SORTIE DE VEHICULE

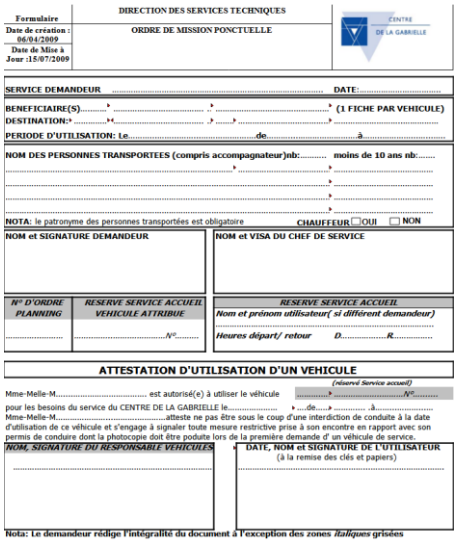
Mme-Melle-M..... est autorisé(e) à utiliser un véhicule de service pour les besoins du service du CENTRE DE LA GABRIELLE du.....au.....de la semaine N°.....

Mme-Melle-M..... atteste ne pas être sous le coup d'une interdiction de conduite pour la période d'utilisation du véhicule et s'engage à signaler toute mesure restrictive prise à son encontre en rapport avec son permis de conduire dont la photocopie doit être produite lors de la première demande d'un véhicule de service.

NOM et SIGNATURE DU RESPONSABLE VEHICULES | DATE, NOM et SIGNATURE DE L'UTILISATEUR

NOTA: Le demandeur rédige l'intégralité du document à l'exception des zones grisées

Photo du formulaire hebdomadaire



Formulaire | DIRECTION DES SERVICES TECHNIQUES
Date de création : 06/04/2009 | ORDRE DE MISSION PONCTUELLE
Date de Mise à Jour :15/07/2009

SERVICE DEMANDEUR ..... DATE:.....

BENEFICIAIRE(S)..... (1 FICHE PAR VEHICULE)
DESTINATION:* .....*.....*.....*.....
PERIODE D'UTILISATION: Le.....de.....à.....

NOM DES PERSONNES TRANSPORTEES (compris accompagnateur)nb:..... moins de 10 ans nb:.....

NOTA: le patronyme des personnes transportées est obligatoire CHAUFFEUR ☐OUI ☐ NON

NOM et SIGNATURE DEMANDEUR | NOM et VISA DU CHEF DE SERVICE

N° D'ORDRE PLANNING	RESERVE SERVICE ACCUEIL VEHICULE ATTRIBUE	RESERVE SERVICE ACCUEIL
.....	.....N°.....	Nom et prénom utilisateur(si différent demandeur) ..... Heures départ/ retour D.....R.....

ATTESTATION D'UTILISATION D'UN VEHICULE
(réservé Service accueil)

Mme-Melle-M..... est autorisé(e) à utiliser le véhicule .....N°.....
pour les besoins du service du CENTRE DE LA GABRIELLE le..... ....de.....à.....
Mme-Melle-M.....atteste ne pas être sous le coup d'une interdiction de conduite à la date d'utilisation de ce véhicule et s'engage à signaler toute mesure restrictive prise à son encontre en rapport avec son permis de conduire dont la photocopie doit être poduite lors de la première demande d' un véhicule de service.

NOM, SIGNATURE DU RESPONSABLE VEHICULES | DATE, NOM et SIGNATURE DE L'UTILISATEUR (à la remise des clés et papiers)

Nota: Le demandeur rédige l'intégralité du document à l'exception des zones *italiques* grisées

Photo du formulaire ponctuelle

I.2.2. Objectifs de l'entreprise

L'entreprise a pour objectif d'améliorer son processus de réservation de véhicules. Elle souhaite répondre à deux besoins essentiels:

- Le premier est de réduire le temps consacré à la gestion des demandes de véhicules.

-Le deuxième est la facilité, remplir un formulaire sur internet est plus simple que de faire une demande de véhicule via papier et de l'envoyer.

Le responsable a eu l'idée de faire un site internet en intranet pour permettre la location de véhicules. Pour lui, c'était plus simple et ergonomique.

II. Missions et projets réalisés

II.1. Ma mission accordée au stage

Le responsable m'a confié une mission : la création d'un site internet intranet pour la réservation de véhicules. Cette responsabilité, discutée lors de notre appel téléphonique avant que je remplisse mon BOS, correspondait à mes demandes et aux demandes attendues de l'université, qui est le développement.

On m'a fourni un bureau et un ordinateur contenant un compte. Cependant, j'ai choisi d'utiliser mon ordinateur personnel car j'avais beaucoup de logiciels pour le développement web.

II.2. Approche de Développement et Choix Technologiques

II.2.1. Consignes et méthodologie de travail

Durant mon stage, le tuteur m'a donné des consignes pour la création du site internet. Il souhaitait que le site soit programmé en PHP et que je respecte l'architecture MVC. De plus, il voulait que je documente mon code pour faciliter d'éventuelles améliorations ultérieures. Les détails concernant la base de données seront précisés ultérieurement.

Pour le site internet, mon tuteur voulait avoir un site simple à utiliser, qui n'a pas beaucoup d'options. Il voulait qu'il y ait 2 rôles (administrateur et utilisateur) et il voulait 2 pages différentes pour chaque

rôle. L’utilisateur peut faire une demande de véhicule, peut voir ses réservations et peut contacter le support technique si nécessaire.

Quant à l’administrateur, il peut voir les réservations des utilisateurs et peut créer des comptes. C’est le tuteur qui donnera les comptes aux employés.

Pour ma préparation, je n'ai pas suivi une méthode Agile ou Scrum, j'ai procédé à une approche itérative. J'ai d'abord voulu analyser la situation, me préparer (poser des questions par rapport au site internet) puis j'ai commencé à le faire.

Je comptais d'abord commencer le projet par le développement front-end. En ayant l’aspect visuel du site internet, je pensais pouvoir créer facilement la base de données ainsi que réaliser la partie back-end. Mon plan était de visualiser les besoins pour établir la base de données.

Suite à cela, je comptais procéder à la conception de la base de données. Travailler sur la base de données après avoir développé le front-end me permettrait facilement de comprendre ce que je pourrais intégrer dans ma base de données.

Enfin, je comptais réaliser le développement back-end. J’avais préféré le faire en dernier car il est plus simple de le connecter avec le front-end et la base de données. Écrire les scripts et les fonctions m’aurait été simple car cela nécessitait l'utilisation de la base de données.

Pour le côté serveur, j’ai utilisé Wamp64. Il est très simple et je le connaissais auparavant. Wamp64 est un environnement de développement web permettant d’exécuter du code PHP, d’intégrer une base de données.

J’ai utilisé Windows 11 car je suis plus à l’aise à utiliser ce système d’exploitation. J’avais déjà tout paramétré sur mon ordinateur donc j’ai opté pour Windows 11.

J’ai utilisé Visual Studio Code pour pouvoir développer et organiser mon code. J’ai choisi spécifiquement Visual Studio Code car il offrait de nombreuses fonctionnalités et une interface conviviale. Cela m’a permis de travailler plus efficacement et de gérer mon projet de manière plus simple.

Et pour finir, j’ai choisi de faire mes designs sur Figma.

J’approfondirai les explications de ces langages / logiciels dans les parties dédiées à ceux-ci.

II.2.2. Outils de conception et de développement (VSCode, Figma)

Durant le projet, j'ai utilisé VSCode et Figma.

VSCode (Visual Studio Code) est un éditeur de code flexible et open-source. Il est très simple à utiliser et comporte plusieurs langages de programmation. C'est très bien car c'est un environnement de développement intégré (IDE) général. Je pourrais développer facilement sans devoir installer d'autres IDE. Par exemple, pour écrire un script PHP, je n'ai qu'à créer un fichier directement sur VSCode. Si je souhaite développer en HTML, je refais la même étape et je pourrais le faire directement dans VSCode sans devoir installer un autre environnement. De même, je pourrais organiser mon dossier de projet dedans.

Figma est un logiciel de design graphique, il permet de faire des designs pour plusieurs types, par exemple un prototypage d'un site internet.

J'ai choisi ces outils parce que je suis très familier avec eux. Je les ai utilisés de nombreuses fois pour différents projets, comme des projets web universitaires pour Figma, VSCode et même en dehors du web.

II.2.3. Organisation du projet et architecture MVC

Avant de commencer le projet, j'ai commencé à organiser un dossier pour pouvoir le faire en localhost, j'ai donc utilisé WAMP64 pour faire le projet. Wamp64 est un environnement de développement web permettant d'exécuter du code PHP et d'intégrer une base de données. C'est comme un serveur. WAMP64 est très simple à utiliser, je n'ai seulement qu'à me connecter à la base de données dans l'adresse « localhost/phpmyadmin ».

phpMyAdmin est une application Web gratuite intégrée dans WAMP64 et open source utilisée pour gérer et administrer les bases de données MySQL et MariaDB. Il permet de créer, de modifier et de supprimer des tables, des lignes et des colonnes dans une base de données, ainsi que d'exécuter des requêtes SQL. C'est comme un tableau de bord.

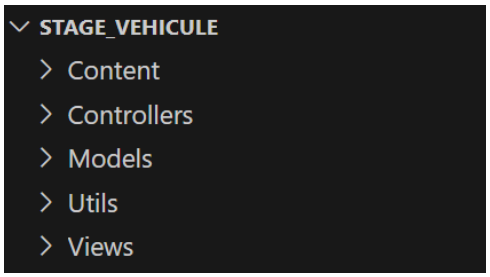
J'ai ensuite commencé à organiser mon projet en respectant l'architecture MVC. L'architecture MVC (Modèle-Vue-Contrôleur) est un modèle d'architecture/de conception contenant trois composants principaux : Model (Modèle), View (Vue) et Controller (Contrôleur). Chaque composant architectural est conçu pour gérer des aspects de développement spécifiques d'une application. Le paradigme de celui-ci est orienté objet.

Le Modèle représente la structure de données, souvent gérée à travers des interactions avec une base de données. Il contient les « données » de l'application, les règles de gestion de ces données, et les fonctions pour interagir avec la base de données et les contrôleurs.

La vue est l'interface graphique de l'application. C'est via cet élément que vont se faire les interactions entre l'utilisateur et le code.

Le contrôleur gère les interactions entre le modèle et la vue. Il reçoit les requêtes de l'utilisateur depuis la vue, traite ces requêtes en fonction des fonctions définies dans le modèle, et ensuite, il renvoie les données à afficher à la vue correspondante.

Voici comment j'ai organisé mon dossier avant de faire le projet :



Dossier du projet

Le dossier « Utils » contient un fichier nommé « credentials.php » ce fichier sert à connecter le site internet et la base de données (voir annexe pour le code).

Le dossier « Content » contiendra des dossiers et des fichiers. Par exemple, pour mettre du style dans une vue, je devrais appliquer un langage de programmation (par exemple CSS). CSS est une feuille de style, elle permet de mettre des styles à la vue. Je compte mettre mes feuilles de styles dans le dossier « Content » avec un sous-dossier « css » car je compte créer un script JavaScript et le placer dans un sous dossier « js ».

II.2.4. Programmation Front-End et maquette

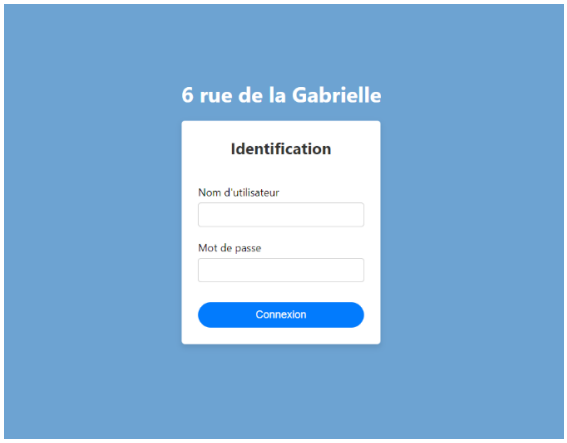
Le front-end est la partie visible de l'utilisateur, c'est tout ce que l'utilisateur pourra voir. Lorsqu'on dit « programmation front-end » c'est l'affichage. Par exemple, mettre des boutons, des images, un formulaire etc…

Après avoir pris en compte le contexte du projet et m'être organisé, j'ai commencé à créer un modèle de maquette avant de passer à la programmation. Tout d'abord, j'ai demandé à M. ANGOT de savoir quel style il souhaitait pour le site. Il m'a demandé de concevoir une interface conviviale et facile à utiliser. Je vais vous montrer étape par étape les maquettes et le développement que j'ai fait.

J'ai d'abord fait la page d'accueil qui est la page de connexion. Je me suis inspiré d'un logiciel open-source de ticketing GestSup. La page d'accueil était la page de connexion car il faudra d'abord se connecter, puis c'est un site intranet il n'était pas nécessaire de faire une page d'accueil.

Voici une photo de la page de connexion GestSup et de la maquette avant de devoir la programmer :

Page de connexion GestSup

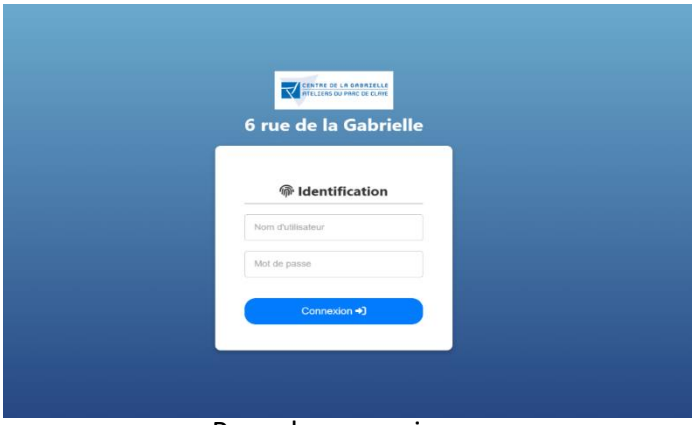


Maquette de la page de connexion

Après avoir montré la page d'accueil, M. ANGOT était plutôt satisfait donc j'ai gardé cette maquette, mais lors du développement j'ai continué à faire une page de connexion ressemblant à celle de GestSup. D'ailleurs, M. ANGOT Connaissait GestSup et était d'accord que je m'inspire de celui-ci pour pouvoir développer le site internet.

J'ai commencé à programmer la vue de la page de connexion. J'ai utilisé du HTML et du CSS. Pour le CSS, j'ai créé un dossier « css » dans le dossier « Content » et j'ai créé un fichier style.css. Avant de pouvoir créer le CSS, j'ai d'abord créé le formulaire de connexion. Le nom du fichier est view_login.php (Voir annexe).

Après avoir programmé la page et la feuille de style, j'ai réussi à faire la page d'accueil qui est la page de connexion.



Page de connexion

La page de connexion est très similaire à celle de GestSup. J'ai opté pour GestSup car c'est un logiciel open-source, donc libre d'utilisation, et son style est plutôt simple. J'ai personnalisé la page avec plusieurs feuilles de styles et j'ai intégré les logos de connexion et d'identification en utilisant une méthode appelée « CDN » permettant de les importer depuis un site internet proposant ces logos, en ne conservant que les liens. Ensuite, j'ai entamé la conception des maquettes pour les vues de début et de fin de l'utilisateur.

La vue « début » regroupe les éléments essentiels que l'utilisateur pourrait avoir besoin dès son arrivée sur le site, comme par exemple de voir ses réservations ou de demander un véhicule. En intégrant cette vue dans plusieurs pages, j'évite de répéter le même code à chaque fois. Cela rend le développement plus efficace. Je ne serais pas obligé d'importer les feuilles de style à chaque fois car en incluant le fichier vue début, le CSS est déjà implémenté dans ce fichier et est donc importé automatiquement. Les importer dans la vue « début » et importer seulement la vue est utile. Pour la vue « fin » j'importerais ce qu'il y a de nécessaire pour la fin de la page, par exemple la fermeture de la page et l'importation du script JavaScript.

Maquette de la vue

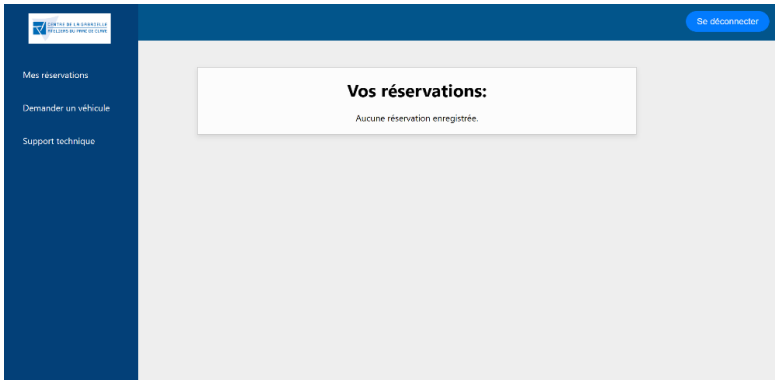
Je me suis inspiré de GestSup et également de GLPI qui est aussi un outil de ticketing comme GestSup.

Le rectangle noir et blanc sera l'affichage des pages. Par exemple, si une personne clique sur mes réservations, ces réservations seront affichées sur le rectangle.

J'ai placé un bouton de déconnexion en haut à droite et à gauche, j'ai mis le logo du Centre de la Gabrielle. J'ai mis les sections à gauche comme GestSup et GLPI.

La section support technique, comme je l'ai dit précédemment, sera simplement une page pour contacter le responsable si nécessaire.

Après avoir fait la vue j'ai directement procédé à la maquette de la vue réservations (view_mes_réservations.php) puis j'ai fait le développement. Voici la vue :



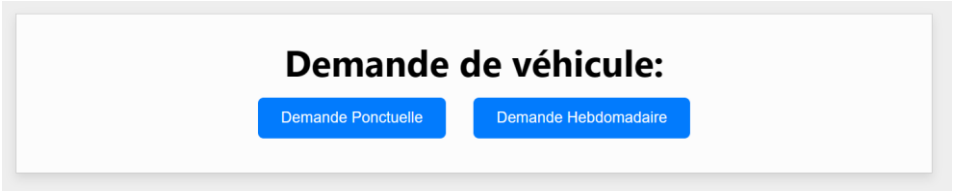
Page « mes réservations » avec la vue début

J'ai ajouté la vue "début" à la page "mes réservations", où apparaît le rectangle à droite affichant "Vos réservations : ...". Je prévois d'adopter une approche similaire pour la section "support technique" ainsi que pour la vue de l'administrateur (vue "voir_demandes").

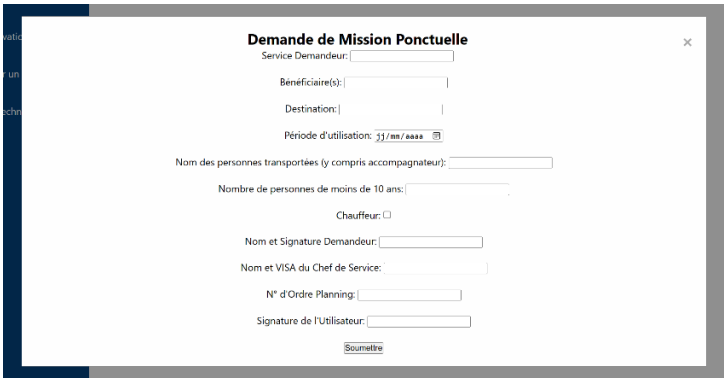
J'ai également poursuivi le développement de la section « support informatique ». Il a fallu ajouter un message pour permettre à toute personne souhaitant contacter le responsable de poser d'éventuelles questions. Je n'ai pas fait de maquette pour celui-ci, j'ai directement commencé à développer.

J'ai procédé au développement de la page « Demander un véhicule ». J'ai ajouté 2 boutons pour faire une demande de véhicule : soit on effectue une demande hebdomadaire, soit une demande ponctuelle, et lorsqu'un utilisateur clique sur un de ces boutons, un modal apparaît. Il pourra remplir le modal et lorsqu'il appuie sur le bouton « envoyer », il sera envoyé à l'administrateur.

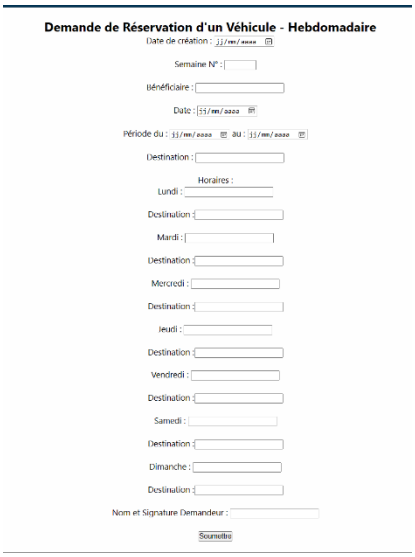
Voici la page ainsi que les modals :



Affichage demande de véhicule



Modal ponctuelle

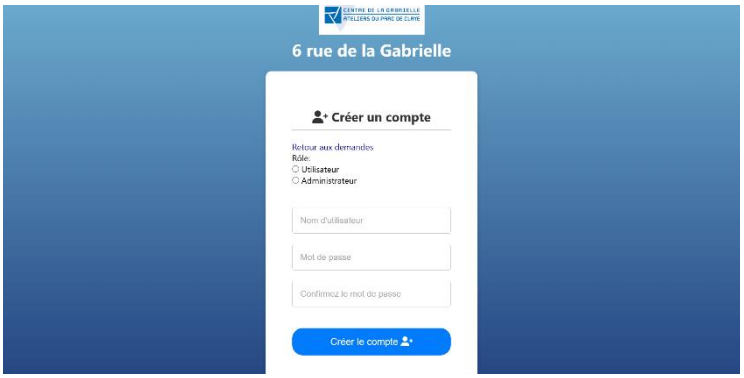


Modal hebdomadaire

Ensuite, j'ai commencé à faire la vue pour l'admin, ainsi que les pages nécessaires de celui-ci. J'ai repris le même d'affichage que l'utilisateur. J'ai seulement changé les sections. L'administrateur a seulement 2 sections : Créer un compte et voir les réservations pour les accepter ou les refuser. Voici la vue de l'administrateur avec la vue « voir les réservations » :

Page de l'administrateur

J'ai utilisé la même « vue début » que celle de l'utilisateur. J'ai simplement créé des vues spécifiques pour l'administrateur. Par exemple, pour la vue début, je l'ai appelé : « view_begin_admin.php ».
Pour la section « créer un compte », l'administrateur sera redirigé vers une page pour créer un compte. J'ai repris le même modèle que celui de la page de connexion. l'interface sera la même que celle de la page de connexion, sauf que je ne la mettrais pas à droite de la vue comme « voir demandes » etc… Je n'ai pas procédé à faire une maquette parce que la page ressemblerait à la page de connexion, et pour moi, il était inutile de faire une maquette pour celle-ci. Voici la page de création de compte :



Page de création de compte

J'ai rajouté un hyperlien « retour aux demandes », lorsque l'admin créera un compte, il aura un message en vert pour indiquer qu'un compte a été créé. L'hyperlien sera juste pour retourner au menu principal de l'administrateur, c'est-à-dire voir les réservations.

Pour le côté JavaScript, j'avais quelques difficultés et j'ai dû chercher sur internet. J'ai seulement ajouté l'ouverture et la fermeture des modals.

II.2.5. Création de la base de données avec programmation Back-End.

Après avoir fini le côté front-end, j'ai procédé à la création de la base de données. J'ai d'abord créé la base de données en l'appelant « bd_lv », puis j'ai commencé à créer un script SQL avant de l'importer. J'ai d'abord créé la table de l'utilisateur. Le voici :

database.sql

```sql
-- Table pour les utilisateurs
CREATE TABLE `utilisateurs` (
  id_utilisateur INT AUTO_INCREMENT PRIMARY KEY,
  nom_utilisateur varchar(50) NOT NULL UNIQUE,
  mot_de_passe varchar(100) NOT NULL,
  roles enum('utilisateur','administrateur') NOT NULL
);
```

Table « utilisateurs »

Cette table contient :

- Id_utilisateur : contient un numéro unique pour chaque utilisateur. Elle augmente automatiquement à chaque nouvel utilisateur et est la clé principale de la table pour identifier les enregistrements de manière unique.
- Nom_utilisateur : Est limité à 50 caractères, qui doit être rempli et unique.
- Mot_de_passe : Est limité à 100 caractères, qui doit aussi être rempli, mais pas unique.
- Roles : Attribue le rôle de l'utilisateur lorsque le compte est créé. Soit il est « utilisateur » soit « administrateur » attribué par l'administrateur.

Ensuite, j'ai commencé à créer la table « demandes » pour voir les demandes et demander. J'ai préféré créer directement une seule table car je trouvais ça plus simple, puis le tuteur voulait une base de données simple. Le fait d'avoir commencé le front-end m'a permis de mieux comprendre ce que je pourrais intégrer dans la base de données, surtout pour la table « demandes ». Ensuite, j'ai ajouté dans le script un code qui permet de « créer » un compte administrateur.

Après avoir créé la base de données, je l'ai intégrée dans le logiciel web phpMyAdmin pour pouvoir utiliser la base de données.

	Table	Action						Lignes	Type	Interclassement	Taille	Perte
☐	demandes		Parcourir	Structure	Rechercher	Insérer	Vider	Supprimer	2	MyISAM	utf8mb4_0900_ai_ci	3,3 kio	-
☐	utilisateurs		Parcourir	Structure	Rechercher	Insérer	Vider	Supprimer	2	MyISAM	utf8mb4_0900_ai_ci	3,2 kio	-
	2 tables	Somme							4	MyISAM	utf8mb4_0900_ai_ci	6,5 kio	0 o

Tables dans phpMyAdmin

Après avoir fait la base de données, j'ai ensuite procédé au développement back-end. Le back-end consiste en l'interaction entre la base de données, le serveur et le front-end. Il n'est pas visible par l'utilisateur.

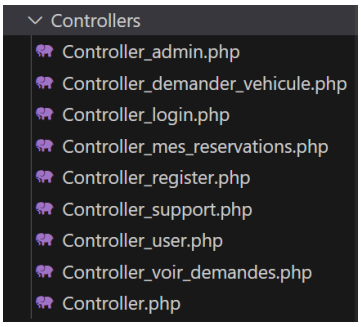
Le back-end gère les données, les traitements, les connexions à la base de données etc. Par exemple, si un utilisateur se connecte à son compte, le fait d'avoir mis en place un back end permettra de communiquer avec la base de données, de vérifier par exemple si le compte existe et donnera l'accès au compte. Bien-sûr, il faudra programmer pour faire fonctionner le lien entre la page de connexion et la base de données.

Par la suite, j'ai développé un fichier nommé index.php. Ce fichier constitue la racine du site internet. Son rôle principal est de diriger les requêtes vers les contrôleurs appropriés. Par exemple, si un utilisateur souhaite consulter ses réservations, il devra accéder au contrôleur spécifié pour la gestion des réservations uniquement, et c'est précisément le rôle de l'index.php que de le diriger vers ce contrôleur correspondant.

Ensuite, j'ai commencé à créer le fichier Controller.php. Il sert à gérer les vues et les associer au contrôleur correspondant. Ce fichier permet aussi d'associer les fonctions du modèle au contrôleur correspondant à la vue.

Enfin, j'ai programmé le fichier Model.php. Le modèle gère les interactions entre la base de données et le site internet. Les requêtes SQL sont effectuées directement dans le modèle.

Suite à cela, j'ai commencé à faire les contrôleurs. Au total j'ai fait 9 contrôleurs :



- Controller_admin : Est le contrôleur de l'administrateur. Il permet d'approuver, de refuser et de voir les demandes faites par les utilisateurs.
- Controller_demander_vehicule : Permet de demander des véhicules.
- Controller_login : Permet de se connecter.
- Controller_mes_reservations : Permet de voir les réservations faites.
- Controller_register : Permet de créer un compte.
- Controller_support Permet de voir la vue support.
- Controller_user : Est le contrôleur de l'utilisateur. Il collecte les données de l'utilisateur et affiche ses réservations.
- Controller_voir_demandes : Permet de voir les demandes effectuées par les utilisateurs.

Au début, j'ai commencé à programmer le contrôleur login, pour essayer de faire fonctionner la page de connexion. J'ai indiqué les messages d'erreur lorsque l'utilisateur se trompe de mot de passe. D'ailleurs, j'ai haché les mots de passe pour garantir une sécurité.

Ensuite, j'ai continué à développer le contrôleur « register ». C'était un peu pareil que le contrôleur « login ». Puis j'ai fait le contrôleur support. Il n'y avait quasiment rien à faire à part vérifier si la personne qui essaye d'accéder à cette vue est un utilisateur et d'afficher la vue.

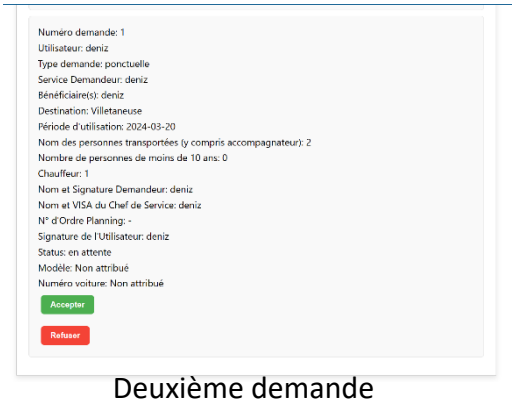
Après, j'ai fait le contrôleur « demander véhicule » pour que l'utilisateur puisse demander un véhicule. Le code était plus long car il y avait plus de choses à mettre, de même pour le modèle.

Et après, j'ai fait le contrôleur « mes réservations ». Il y avait aussi beaucoup de fonctions à implémenter. Après avoir implémenté les fonctions, j'ai fait un test. J'ai effectué 2 demandes (ponctuelle et hebdomadaire). Voici ce qu'on retrouve après avoir fait les demandes :

Affichage après avoir effectué 2 demandes

Ensuite, j'ai procédé à faire le développement du contrôleur « voir demandes ». Il était aussi long à faire parce qu'il y avait beaucoup de fonctions à implémenter, et je devais afficher uniquement les informations dédiées aux types de formulaires. Après avoir fini de programmer, j'ai effectué un test. Je me suis connecté dans un compte administrateur pour voir les demandes que j'ai effectuées auparavant. Voici ce qu'on retrouve :

Première demande



Deuxième demande

Lorsque je clique sur « accepter », j'ai un petit champ de texte qui s'affiche. Je dois attribuer le modèle d'un véhicule et un numéro de véhicule (le numéro de véhicule a une identification unique, il permet d'identifier le véhicule) et lorsque j'accepte, les données entrées sont prises en compte et les valeurs 'status', 'modèle' et 'numéro voiture' changent. Lorsque je refuse, aucun champ de texte n'apparaît. Juste le 'status' change en 'refusée'.

J'ai ensuite procédé au développement du contrôleur « user » et « admin ». Ces contrôleurs sont quasiment identiques, sauf que le contrôleur « admin » a plus de fonctions (approuver les demandes, les décliner et voir les demandes).

Le contrôleur user permet d'obtenir les réservations par l'id de l'utilisateur. Voici le code de controller_user :

```php
<?php

class Controller_user extends Controller {

    public function action_default() {
        $this->action_user();
    }

    public function action_user() {
        // Assurez-vous que l'utilisateur est connecté
        if (!isset($_SESSION['id_utilisateur'])) {
            header('Location: /stage_vehicule/index.php');
            exit();
        }

        $id_utilisateur = $_SESSION['id_utilisateur'];

        // Obtenir une instance du modèle
        $m = Model::getModel();

        // Récupérer les réservations de l'utilisateur
        $reservations = $m->getReservationsByUserId($id_utilisateur);
    }

}
```

Code controller_user

Il y a deux fonctions, getReservationsByUserId et action_user. La fonction getReservationsByUserId est dans le modèle. Action_user est la fonction pour le controller_user.

Voici la fonction getReservationsByUserId dans le modèle :

```php
73  public function getReservationsByUserId($id_utilisateur) {
74      // Assurez-vous d'avoir une connexion à la base de données ($this->db)
75      $sql = "SELECT id_demande, status, modele_voiture, numero_voiture, type_demande FROM demandes WHERE id_utilisateur = :id_utilisateur ORDER BY date_creation DESC";
76
77      try {
78          $stmt = $this->db->prepare($sql);
79          $stmt->bindParam(':id_utilisateur', $id_utilisateur, PDO::PARAM_INT);
80          $stmt->execute();
81
82          return $stmt->fetchAll(PDO::FETCH_ASSOC);
83      } catch (Exception $e) {
84          // Gestion d'erreur
85          error_log('Erreur lors de la récupération des réservations : ' . $e->getMessage());
86          return [];
87      }
88  }
```

Partie d'un code modèle

La fonction prépare la requête SQL qui est de sélectionner les demandes (réservations) de l'utilisateur. Je prépare les requêtes pour éviter les injections SQL. J'ai utilisé try et catch ici pour éviter que le site web ait une erreur s'il y a une erreur. Si le catch est exécuté, le site internet va toujours fonctionner.

Après avoir fini de développer les contrôleurs, j'ai commencé à faire un script logout.php pour se déconnecter lorsqu'on est connecté. J'ai seulement initialisé la session, la désactivé puis la détruit. Chaque compte connecté à une session démarré.

II.2.6. Résultats du site internet

Après avoir fini de tout développer, j'ai testé le site internet pour voir si tout fonctionnait correctement. Après les tests, j'ai rencontré quelques problèmes, mais j'ai pu les corriger (formulaire hebdomadaire qui ne voulait pas s'envoyer etc…). Le site est opérationnel, mais n'a pas été mis en ligne sur l'intranet.

La base de données fonctionne correctement. J'ai un petit peu sécurisé le site internet en employant des méthodes dans les fonctions pour éviter les attaques de type Cross-Site Scripting.

J'ai d'ailleurs testé des injections SQL sur le site internet pour voir si le site était bien protégé et c'est le cas dû à la préparation de requête lors des données entrées dans la base de données.

III. Ma première expérience professionnelle

III.1. Challenges rencontrés

J'ai rencontré plusieurs difficultés lors de mon stage, la première difficulté étant le trajet. Je mettais plus d'1h30 de trajet en général, c'était rare que je mette moins d'1h30. Je n'étais au début pas habitué à faire tout ce trajet. J'ai même réfléchi à changer de stage. Mais au fil du temps, je me suis habitué à faire ces trajets.

La deuxième difficulté était la création de la base de données. Lors du développement de la base de données, je ne savais pas réellement comment la structurer. J'ai essayé plusieurs méthodes et au final, j'ai réussi.

La dernière difficulté que j'ai eue était le back-end. À l'université, je n'avais pas un bon niveau pour faire le back-end, j'avais surtout du mal avec les contrôleurs. J'ai regardé beaucoup de vidéos lorsque je développais et je cherchais beaucoup sur internet. Je m'aidais aussi sur internet.

III.2. Mon avis sur cette expérience

J'ai aimé cette expérience sachant que c'était ma première expérience. Je n'ai jamais effectué un stage ou un travail dans une équipe informatique et n'ai jamais eu des tâches liées à ce domaine. J'avais l'impression que j'étais embauché dans une entreprise avec un contrat, car je n'allais plus en cours (2 mois). Le fait de faire de longs trajets et le fait que je ne connaissais quasiment personne dans l'établissement m'a donné cette sensation.

J'ai pu améliorer mes compétences liées au web et comprendre comment fonctionnaient les contrôleurs et les modèles.

Conclusion

Pour conclure, J'ai aimé ce stage, j'ai pu découvrir beaucoup de choses, le fait de savoir comment était le « monde du travail », j'ai pu améliorer mes compétences, que ce soit hard skill ou soft skill, comme par exemple savoir comment s'organiser pour un projet, car ce n'était pas un devoir universitaire, mais un vrai projet. Le fait de s'organiser était différent. J'ai bien aimé cette expérience, j'ai pu aussi améliorer mes compétences en autonomie, comme le fait de chercher sur internet pour pouvoir résoudre mes problèmes.

Pour mes compétences hard skills, j'ai pu apprendre, comme dit précédemment, la compréhension du langage PHP et de l'architecture MVC. J'ai aussi appris des choses que je ne connaissais pas durant mon université, qui sont les CDN et les modals.

J'ai également pu découvrir le travail en tant que développeur full-stack, et ce domaine ne m'a l'air pas intéressant. Je n'ai pas aimé le fait de faire que de programmer, mais je suis content d'avoir pu faire cette expérience car j'ai pu découvrir le métier du développement web.

Je n'ai cependant rien à dire sur le lieu de travail. J'ai bien aimé le lieu de travail, c'était calme et on n'était pas beaucoup et j'aime bien lorsqu'il y a peu de personnes.

J'ai approfondi la compétence 1 qui est la réalisation d'un développement d'applications. J'ai approfondi les langages de programmation que j'ai utilisé tels que PHP, HTML et CSS.

Glossaire

- **Développeur Full-stack :** Un développeur full-stack est un développeur qui fait à la fois du développement front-end et du développement back-end.

- **PHP :** (Hypertext Preprocessor) est un langage de programmation conçu pour le côté serveur dans le contexte du web.

- **IDE :** (Integrated Development Environment) est un environnement de programmation sous forme d'une application. Il contient les outils pour le langage de programmation en question.

- **HTML :** (HyperText Markup Language) est un langage de balisage. Il permet de structurer un site internet en utilisant des balises telles que le titre, un champ de texte etc…

- **Open-Source :** C'est un logiciel (ou autre) libre de droit, où le code source est accessible à tous. Cela permet à chacun de visualiser, de modifier et de télécharger le logiciel.

- **CDN :** (Content Delivery Network) est un réseau de serveurs interconnectés qui permet de distribuer des contenus web tel que des images, des logos etc…

- **JavaScript :** C'est un langage de programmation web qui permet d'ajouter des fonctionnalités dynamiques à des sites web.

- **GLPI :** (Gestionnaire Libre de Parc Informatique) c'est un logiciel open-source de gestion des services informatiques et de gestion des services d'assistance.

- **Modal :** fenêtre qui s'ouvre dans une page web permettant d'interagir avec celui-ci. Elle peut demander une action de l'utilisateur ou autre.

- **Hyperlien :** Lien dans un site web qui permet de rendre l'utilisateur à cette destination.

- **SQL :** (Structured Query Language) est un langage de programmation dédié aux bases de données.

- **Clé principale (SQL) :** Identifie un enregistrement de manière unique d'une table.

- **Requête SQL :** Permet d'interroger une base de données.

- **Intranet :** C'est un réseau informatique privé utilisé par une entreprise ou autre, accessible uniquement aux membres et non au public.

- **Cross-Site Scripting (XSS) :** Vulnérabilité consistant à insérer un code malveillant dans des pages web pouvant affecter les utilisateurs qui les consultent.

- **Injection SQL :** Vulnérabilité consistant à insérer du code SQL malveillant dans le but d'accéder ou de manipuler les données stockées dans la base de données.

Annexes

Voici les annexes. Il y a des scripts où je n'ai pas mis l'entièreté car il y en a qui sont longs.

Code index.php :

```php
<?php

// Start the session at the beginning of the script
session_start();

// Set error reporting for development (turn off error reporting in production)
ini_set('display_errors', 1);
ini_set('display_startup_errors', 1);
error_reporting(E_ALL);

// Include the base Model and Controller classes
require_once 'Models/Model.php';
require_once 'Controllers/Controller.php';

// Include database credentials
include_once "Utils/credentials.php";

// List of controllers available in the application
$controllers = ["mes_reservations", "register", "demander_vehicule", "login", "user", "support", "voir_demandes", "admin"];

// Default controller if none is specified or if the specified controller is not in the list
$controller_default = "login";

// Check if the controller exists and is in the list of controllers
if (isset($_GET['controller']) and in_array($_GET['controller'], $controllers)) {
    $nom_controller = $_GET['controller'];
} else {
    $nom_controller = $controller_default;
}

// Determine the name of the controller class
$nom_classe = 'Controller_' . $nom_controller;

// Determine the path to the controller file
$nom_fichier = 'Controllers/' . $nom_classe . '.php';

// Check if the file exists and is readable
if (is_readable($nom_fichier)) {
    // Include the controller file and instantiate the controller class
    include_once $nom_fichier;
    $controller = new $nom_classe($nom_controller);
} else {
    // If the controller file doesn't exist, display an error message
    die("Error 404: not found!");
}

// À ajouter dans index.php après l'instanciation du contrôleur
if (isset($_GET['action'])) {
    $action = $_GET['action'];
    if (method_exists($controller, 'action_' . $action)) {
        $controller->{'action_' . $action}();
    } else {
        // Gérer l'action inexistante, par exemple en appelant l'action par défaut
        $controller->action_default();
    }
} else {
    $controller->action_default();
}
```

Code logout.php :

```php
<?php

session_start(); //Init la session
session_unset(); //Desactiver la session
session_destroy();//Detruit la session
header("location: index.php");//redirection

?>
```

Code credentials.php :

```php
<?php
    $dsn = 'mysql:host=localhost;dbname=bd_lv';
    $login = 'root';
    $mdp = '';
?>
```

Code database.sql :

```sql
-- Table pour les utilisateurs
CREATE TABLE `utilisateurs` (
  id_utilisateur INT AUTO_INCREMENT PRIMARY KEY,
  nom_utilisateur varchar(50) NOT NULL UNIQUE,
  mot_de_passe varchar(100) NOT NULL,
  roles enum('utilisateur','administrateur') NOT NULL
);


CREATE TABLE demandes (
  id_demande INT AUTO_INCREMENT PRIMARY KEY,
  id_utilisateur INT NOT NULL,
  type_demande ENUM('ponctuelle', 'hebdomadaire') NOT NULL,
  status ENUM('en attente', 'confirmée', 'refusée') NOT NULL DEFAULT 'en attente',
  date_creation DATETIME DEFAULT CURRENT_TIMESTAMP,
  service_demandeur VARCHAR(50),
  beneficiaire VARCHAR(255),
  destination VARCHAR(255),
  periode_utilisation DATE,
  periode_utilisation_de DATE, -- Pour les demandes hebdomadaires, cela représente la période du début
  periode_utilisation_a DATE, -- Pour les demandes hebdomadaires, cela représente la période de fin
  nom_transportes TEXT,
  nb_moins_10_ans INT DEFAULT 0,
  chauffeur BOOLEAN DEFAULT FALSE,
  nom_demandeur VARCHAR(50),
  visa_chef_service VARCHAR(50), -- Ajouté basé sur le formulaire
  no_ordre_planning VARCHAR(50), -- Ajouté basé sur le formulaire
  signature_utilisateur TEXT, -- Ajouté basé sur le formulaire
  semaine_no INT, -- Uniquement pour les demandes hebdomadaires
  date_hebdo DATE, -- Uniquement pour les demandes hebdomadaires
  horaires TEXT, -- Pour stocker les horaires de façon flexible. Vous pourriez envisager une structure JSON pour plus de flexibilité
  modele_voiture VARCHAR(50) DEFAULT NULL,
  numero_voiture INT DEFAULT NULL,
  lundi VARCHAR(20),
  destination_lundi VARCHAR(20),
  mardi VARCHAR(20),
  destination_mardi VARCHAR(20),
  mercredi VARCHAR(20),
  destination_mercredi VARCHAR(20),
  jeudi VARCHAR(20),
  destination_jeudi VARCHAR(20),
  vendredi VARCHAR(20),
  destination_vendredi VARCHAR(20),
  samedi VARCHAR(20),
  destination_samedi VARCHAR(20),
  dimanche VARCHAR(20),
  destination_dimanche VARCHAR(20),
  beneficiaire_hebdo VARCHAR(20),
  destination_hebdo VARCHAR(20),
  nom_demandeur_hebdo VARCHAR(20),
  FOREIGN KEY (id_utilisateur) REFERENCES utilisateurs(id_utilisateur)
);

INSERT INTO utilisateurs (nom_utilisateur, mot_de_passe, roles)
VALUES ([redacted]
```

Code view_begin.php :

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Accueil</title>
    <link rel="stylesheet" href="/stage_vehicule/Content/css/style.css">
</head>
<body>
    <div class="view-home">
        <aside class="sidebar">
            <div class="logo">
                <img src="/stage_vehicule/Content/images/logo.png" alt="Logo">
            </div>
            <nav class="nav-menu">
            <a href="?controller=mes_reservations" class="nav-item">Mes réservations</a>
            <a href="?controller=demander_vehicule&action=default" class="nav-item">Demander un véhicule</a>
            <a href="?controller=support" class="nav-item">Support technique</a>

            </nav>
        </aside>
        <div class="main-content">
            <header class="top-header">
                <button class="profile-btn" onclick="location.href='logout.php';">Se déconnecter</button>
            </header>
            <div class="content-area">
        </div>
```

Code view_begin_admin.php :

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Accueil</title>
    <link rel="stylesheet" href="/stage_vehicule/Content/css/style.css">
</head>
<body>
    <div class="view-home">
        <aside class="sidebar">
            <div class="logo">
                <img src="/stage_vehicule/Content/images/logo.png" alt="Logo">
            </div>
            <nav class="nav-menu">
                <a href="?controller=voir_demandes" class="nav-item">Voir les réservations</a>
                <a href="?controller=register&action=signup" class="nav-item">Créer un utilisateur</a>
            </nav>
        </aside>
        <div class="main-content">
            <header class="top-header">
                <button class="profile-btn" onclick="location.href='logout.php';">Se déconnecter</button>
            </header>
            <div class="content-area">
```

Code view_end.php & view_end_admin.php :

```html
</div>
        </div>
    </div>
    <script src="/stage_vehicule/Content/js/script.js"></script>
</body>
</html>
```

Code view_login.php :

```html
<!DOCTYPE html>
<html lang="fr">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Login Page</title>
    <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.0.0-beta3/css/all.min.css">
    <link rel="stylesheet" href="/stage_vehicule/Content/css/style.css">
</head>
<body class="body-login">
    <div class="login-container">
        <header class="top-bar">
            <h2><img src="/stage_vehicule/Content/images/logo.png" width="200"></h2>
            <h1>6 rue de la Gabrielle</h1>
        </header>
        <section class="login-box">
            <h2><i class="fa-solid fa-fingerprint"></i>  Identification</h2>
            <?php
            echo isset($errorLogin) ? $errorLogin : '';
            echo isset($successLogin) ? $successLogin : '';
            ?>
            <form class="login-form" method="post" action="?controller=login&action=login">
            <div class="underline"></div>
                <div class="input-group">
                    <input type="text" id="username" name="username" required placeholder="Nom d'utilisateur">
                </div>
                <div class="input-group">
                    <input type="password" id="password" name="password" required placeholder="Mot de passe">
                </div>
                <button type="submit" class="login-btn" id="l">
                    Connexion <i class="fa-solid fa-right-to-bracket"></i>
                </button>
            </form>
        </section>
    </div>
    <script src="/stage_vehicule/Content/js/script.js"></script>
</body>
</html>
```

Code view_register.php :

```php
<!DOCTYPE html>
<html lang="fr">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Page d'inscription</title>
    <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.0.0-beta3/css/all.min.css">
    <link rel="stylesheet" href="/stage_vehicule/Content/css/style.css">
</head>
<body class="body-login">
    <div class="login-container">
        <header class="top-bar">
            <h2><img src="/stage_vehicule/Content/images/logo.png" width="200"></h2>
            <h1>6 rue de la Gabrielle</h1>
        </header>
        <section class="login-box">
        <?php if (!empty($successMsg)): ?>
            <div style="color: green;"><?php echo $successMsg; ?></div>
        <?php endif; ?>
        <?php if (!empty($errorUser)): ?>
            <div style="color: red;"><?php echo $errorUser; ?></div>
        <?php endif; ?>
        <?php if (!empty($errorMdp)): ?>
            <div style="color: red;"><?php echo $errorMdp; ?></div>
        <?php endif; ?>
        <?php if (!empty($errorCreation)): ?>
            <div style="color: red;"><?php echo $errorCreation; ?></div>
        <?php endif; ?>
            <h2><i class="fa-solid fa-user-plus"></i>  Créer un compte</h2>
            <div class="underline"></div>
            <form class="login-form" method="post" action="?controller=register&action=signup">
                <div class="role-selection">
                    <!-- Bouton de retour à la page des demandes -->
                <div class="back-to-home">
                    <a href="?controller=voir_demandes&action=default" class="btn-back">Retour aux demandes</a>
                </div>

                    <label>Rôle:</label>
                    <br>
                    <input type="radio" id="utilisateur" name="role" value="utilisateur" required>
                    <label for="utilisateur">Utilisateur</label><br>
                    <input type="radio" id="administrateur" name="role" value="administrateur" required>
                    <label for="administrateur">Administrateur</label>
                </div>
                <br>
                <div class="input-group">
                    <input type="text" id="username" name="username" required placeholder="Nom d'utilisateur">
                </div>
                <div class="input-group">
                    <input type="password" id="password" name="password" required placeholder="Mot de passe">
                </div>
                <div class="input-group">
                    <input type="password" id="password_c" name="password_c" required placeholder="Confirmez le mot de passe">
                </div>

                <button type="submit" class="login-btn" id="register">
                    Créer le compte <i class="fa-solid fa-user-plus"></i>
                </button>
            </form>
        </section>
    </div>
    <script src="/stage_vehicule/Content/js/script.js"></script>
</body>
</html>
```

Code view_voir_demandes.php :

```php
<?php include 'view_begin_admin.php'; ?>

<div class="content-area">
    <div class="voir_demandes">
        <h1>Demandes des utilisateurs:</h1>
        <div class="demandes-container">
            <?php
            if (!empty($demandes)) {
                foreach ($demandes as $demande) {
                    echo "<div class='voir-demandes-card'>";
                    echo "<div class='voir-demandes-details'>";
                    echo "<p>Numéro demande: " . htmlspecialchars($demande['id_demande']) . "</p>";
                    echo "<p>Utilisateur: " . htmlspecialchars($demande['nom_utilisateur']) . "</p>"; // Assurez-vous que 'nom_utilisateur' est correctement récupéré et joint à votre requête SQL
                    echo "<p>Type demande: " . htmlspecialchars($demande['type_demande']) . "</p>";
                    if ($demande['type_demande'] === 'ponctuelle') {
                        // Affichez les informations spécifiques à la demande ponctuelle
                        echo "<p>Service Demandeur: " . (isset($demande['service_demandeur']) ? htmlspecialchars($demande['service_demandeur']) : 'Non disponible') . "</p>";
                        echo "<p>Bénéficiaire(s): " . (isset($demande['beneficiaire']) ? htmlspecialchars($demande['beneficiaire']) : 'Non disponible') . "</p>";
                        echo "<p>Destination: " . (isset($demande['destination']) ? htmlspecialchars($demande['destination']) : 'Non disponible') . "</p>";
                        echo "<p>Période d'utilisation: " . (isset($demande['periode_utilisation']) ? htmlspecialchars($demande['periode_utilisation']) : 'Non disponible') . "</p>";
                        echo "<p>Nom des personnes transportées (y compris accompagnateur): " . (isset($demande['nom_transportes']) ? htmlspecialchars($demande['nom_transportes']) : 'Non disponible') . "</p>";
                        echo "<p>Nombre de personnes de moins de 10 ans: " . (isset($demande['nb_moins_10_ans']) ? htmlspecialchars($demande['nb_moins_10_ans']) : 'Non disponible') . "</p>";
                        echo "<p>Chauffeur: " . (isset($demande['chauffeur']) ? htmlspecialchars($demande['chauffeur']) : 'Non disponible') . "</p>";
                        echo "<p>Nom et Signature Demandeur: " . (isset($demande['nom_demandeur']) ? htmlspecialchars($demande['nom_demandeur']) : 'Non disponible') . "</p>";
                        echo "<p>Nom et VISA du Chef de Service: " . (isset($demande['visa_chef_service']) ? htmlspecialchars($demande['visa_chef_service']) : 'Non disponible') . "</p>";
                        echo "<p>N° d'Ordre Planning: " . (isset($demande['no_ordre_planning']) ? htmlspecialchars($demande['no_ordre_planning']) : 'Non disponible') . "</p>";
                        echo "<p>Signature de l'Utilisateur: " . (isset($demande['signature_utilisateur']) ? htmlspecialchars($demande['signature_utilisateur']) : 'Non disponible') . "</p>";
                    } else if ($demande['type_demande'] === 'hebdomadaire') {
                        // Affichez les informations spécifiques à la demande hebdomadaire
                        echo "<p>Semaine N° : " . (isset($demande['semaine_no']) ? htmlspecialchars($demande['semaine_no']) : 'Non disponible') . "</p>";
                        echo "<p>Date de création : " . (isset($demande['date_creation_hebdo']) ? htmlspecialchars($demande['date_creation_hebdo']) : 'Non disponible') . "</p>";
                        echo "<p>Bénéficiaire : " . (isset($demande['beneficiaire_hebdo']) ? htmlspecialchars($demande['beneficiaire_hebdo']) : 'Non disponible') . "</p>";
                        echo "<p>Date : " . (isset($demande['date_hebdo']) ? htmlspecialchars($demande['date_hebdo']) : 'Non disponible') . "</p>";
                        echo "<p>Période du : " . (isset($demande['periode_utilisation_de']) ? htmlspecialchars($demande['periode_utilisation_de']) : 'Non disponible') . " au : " . (isset($demande['periode_utilisation_a']) ? htmlspecialchars($demande['periode_utilisa
                        echo "<p>Destination : " . (isset($demande['destination_hebdo']) ? htmlspecialchars($demande['destination_hebdo']) : 'Non disponible') . "</p>";
                        echo "<p>Horaires : Lundi : " . (isset($demande['lundi']) ? htmlspecialchars($demande['lundi']) . " Destination : " . (isset($demande['destination_lundi']) ? htmlspecialchars($demande['destination_lundi']) : 'Non disponible') : 'Non disponible
                        echo "<p>Nom et Signature Demandeur : " . (isset($demande['nom_demandeur_hebdo']) ? htmlspecialchars($demande['nom_demandeur_hebdo']) : 'Non disponible') . "</p>";
                    }
                    echo "<p>Status: " . htmlspecialchars($demande['status']) . "</p>";
                    echo "<p>Modèle: " . htmlspecialchars($demande['modele_voiture'] ?? 'Non attribué') . "</p>";
                    echo "<p>Numéro voiture: " . htmlspecialchars($demande['numero_voiture'] ?? 'Non attribué') . "</p>";


                    // Formulaire pour approuver la demande
                    // Formulaire pour approuver la demande
                    if ($demande['status'] === 'en attente') {
                        echo "<form action='?controller=voir_demandes&action=approuver_demande' method='post' id='form-accepter-" . $demande['id_demande'] . "' style='display: none;'>";
                        echo "<input type='hidden' name='id_demande' value='" . $demande['id_demande'] . "'>";
                        echo "<input type='text' name='modele_voiture' placeholder='Modèle de voiture'>";
                        echo "<input type='text' name='numero_voiture' placeholder='Numéro de voiture'>";
                        echo "<button type='submit' class='btn-action btn-accepter'>Attribuer</button>";
                        echo "</form>";
                        echo "<button onclick='document.getElementById(\"form-accepter-" . $demande['id_demande'] . "\").style.display = \"block\";' class='btn-action btn-accepter'>Accepter</button>";
                    }

                    // Formulaire pour refuser la demande
                    if ($demande['status'] === 'en attente') {
                        echo "<form action='?controller=voir_demandes&action=refuser_demande' method='post'>";
                        echo "<input type='hidden' name='id_demande' value='" . $demande['id_demande'] . "'>";
                        echo "<button type='submit' class='btn-action btn-refuser'>Refuser</button>";
                        echo "</form>";
                    }


                    echo "</div>"; // Fin .voir-demandes-details
                    echo "</div>"; // Fin .voir-demandes-card
                }
            } else {
                echo "<p>Aucune demande enregistrée.</p>";
            }
            ?>
        </div>
    </div>
</div>
```

Code view_demander_vehicule.php :

```php
<?php include 'view_begin.php'; ?>

<div class="content-area">
    <div class="demander_vehicule">
        <h1>Demande de véhicule:</h1>
        <button id="demandePonctuelleBtn">Demande Ponctuelle</button>
        <button id="demandeHebdomadaireBtn">Demande Hebdomadaire</button>

        <!-- Pop-up Formulaires -->

        <!-- Contenu du formulaire ponctuelle -->

        <div id="demandePonctuelleModal" class="modal">
        <div class="modal-content">
        <span class="close">&times;</span>
        <form action="?controller=demander_vehicule&action=demander_vehicule" method="post">
            <h2>Demande de Mission Ponctuelle</h2>
            <label for="serviceDemandeur">Service Demandeur:</label>
            <input type="text" id="serviceDemandeur" name="serviceDemandeur" required><br><br>

            <label for="beneficiaire">Bénéficiaire(s):</label>
            <input type="text" id="beneficiaire" name="beneficiaire" required><br><br>

            <label for="destination">Destination:</label>
            <input type="text" id="destination" name="destination" required><br><br>

            <label for="periodeUtilisation">Période d'utilisation:</label>
            <input type="date" id="periodeUtilisation" name="periodeUtilisation" required><br><br>

            <label for="nomTransportes">Nom des personnes transportées (y compris accompagnateur):</label>
            <input type="text" id="nomTransportes" name="nomTransportes"><br><br>

            <label for="nbMoins10Ans">Nombre de personnes de moins de 10 ans:</label>
            <input type="number" id="nbMoins10Ans" name="nbMoins10Ans" min="0"><br><br>

            <label for="chauffeur">Chauffeur:</label>
            <input type="checkbox" id="chauffeur" name="chauffeur"><br><br>

            <label for="nomDemandeur">Nom et Signature Demandeur:</label>
            <input type="text" id="nomDemandeur" name="nomDemandeur" required><br><br>

            <label for="visaChefService">Nom et VISA du Chef de Service:</label>
            <input type="text" id="visaChefService" name="visaChefService" required><br><br>

            <label for="noOrdrePlanning">N° d'Ordre Planning:</label>
            <input type="text" id="noOrdrePlanning" name="noOrdrePlanning"><br><br>

            <label for="signatureUtilisateur">Signature de l'Utilisateur:</label>
            <input type="text" id="signatureUtilisateur" name="signatureUtilisateur"><br><br>
            <input type="hidden" name="type_demande" value="ponctuelle">
            <input type="submit" value="Soumettre">
        </form>
    </div>
</div>


        <div id="demandeHebdomadaireModal" class="modal">
        <div class="modal-content">
            <!-- Contenu du formulaire hebdomadaire -->
        <span class="close">&times;</span>
        <form action="?controller=demander_vehicule&action=demander_vehicule" method="post">
            <h2>Demande de Réservation d'un Véhicule - Hebdomadaire</h2>
            <!-- Pour le formulaire hebdomadaire -->
            <label for="dateCreationHebdo">Date de création :</label>
            <input type="date" id="dateCreationHebdo" name="dateCreationHebdo"><br><br>

            <label for="semaineNo">Semaine N° :</label>
            <input type="number" id="semaineNo" name="semaineNo" min="1" max="52"><br><br>

            <label for="beneficiaireHebdo">Bénéficiaire :</label>
            <input type="text" id="beneficiaireHebdo" name="beneficiaireHebdo" required><br><br>

            <label for="dateHebdo">Date :</label>
```

Code view_mes_reservations.php :

```php
<?php include 'view_begin.php'; ?>

<div class="content-area">
    <div class="mes_reservations">
        <h1>Vos réservations:</h1>
        <div class="reservation-container">
            <?php
            if (!empty($reservations)) {
                foreach ($reservations as $reservation) {
                    echo "<div class='reservation-card'>";
                    echo "<div class='reservation-details'>";
                    echo "<p>Numéro demande: " . htmlspecialchars($reservation['id_demande']) . "</p>";
                    echo "<p>Type demande: " . htmlspecialchars($reservation['type_demande'] ?: '-') . "</p>";
                    echo "<p>Status: " . htmlspecialchars($reservation['status']) . "</p>";
                    // Assuming 'modele_voiture' and 'numero_voiture' are the columns for car model and number
                    echo "<p>Modèle: " . htmlspecialchars($reservation['modele_voiture'] ?: '-') . "</p>";
                    echo "<p>Numéro voiture: " . htmlspecialchars($reservation['numero_voiture'] ?: '-') . "</p>";
                    echo "</div>";
                    echo "</div>";
                }
            } else {
                echo "<p>Aucune réservation enregistrée.</p>";
            }
            ?>
        </div>
    </div>
</div>

<?php include 'view_end.php'; ?>
```

Code view_support.php :

```php
<?php include 'view_begin.php' ?>

<div class="content-area">
    <div class="mes_reservations">
    Pour toute question ou problème technique concernant le site, veuillez contacter n
    </div>
</div>

<?php include 'view_end.php' ?>
```

Code Model.php :

```php
<?php

class Model {
    private $db;
    private static $instance = null;

    private function __construct() {
        include __DIR__ . '/../Utils/credentials.php'; // Corrigez le chemin ici
        $this->db = new PDO($dsn, $login, $mdp);
        $this->db->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
    }

    public static function getModel() {
        if (self::$instance === null) {
            self::$instance = new Model();
        }
        return self::$instance;
    }


/* Inscription */


    public function createUser($username, $password, $role) {
        $password = password_hash($password, PASSWORD_DEFAULT);
        $req = $this->db->prepare('INSERT INTO utilisateurs (nom_utilisateur, mot_de_passe, roles) VALUES (?, ?, ?)');
        $req->execute([$username, $password, $role]);
        return $this->db->lastInsertId();
    }

    public function usernameExists($username) {
        $req = $this->db->prepare('SELECT COUNT(*) FROM utilisateurs WHERE nom_utilisateur = ?');
        $req->execute([$username]);
        return $req->fetchColumn() > 0;
    }

/* Connexion */

    public function login($username, $password) {
        $req = $this->db->prepare('SELECT id_utilisateur, nom_utilisateur, mot_de_passe, roles FROM utilisateurs WHERE nom_utilisateur = ?');
        $req->execute([$username]);
        $user = $req->fetch(PDO::FETCH_ASSOC);

        if ($user && password_verify($password, $user['mot_de_passe'])) {
            return $user; // Retourne toutes les infos de l'utilisateur, y compris son rôle
        } else {
            return false;
        }
    }

    public function findUserByUsername($username) {
        $req = $this->db->prepare('SELECT * FROM utilisateurs WHERE nom_utilisateur = ?');
        $req->execute([$username]);
        return $req->fetch();
    }
    public function getUserAndRole($nom_utilisateur) {
        $req = $this->db->prepare("
            SELECT *,
                CASE roles
                    WHEN 'administrateur' THEN 'admin'
                    WHEN 'utilisateur' THEN 'utilisateur'
                    ELSE 'inconnu'
                END as role
            FROM utilisateurs
            WHERE nom_utilisateur = ?"
        );
        $req->execute([$nom_utilisateur]);
        return $req->fetch(PDO::FETCH_ASSOC);
    }

/* Mes réservations */

public function getReservationsByUserId($id_utilisateur) {
    // Assurez-vous d'avoir une connexion à la base de données ($this->db)
```

Code Controller.php :

```php
<?php

abstract class Controller
{
    public function __construct()
    {
        if (isset($_GET['action']) and method_exists($this, "action_" . $_GET["action"])) {
            $action = "action_" . $_GET["action"];
            $this->$action();
        } else {
            $this->action_default();
        }
    }

    abstract public function action_default();

    protected function render($vue, $data = [])
    {
        extract($data);
        $file_name = "Views/view_" . $vue . '.php';
        if (file_exists($file_name)) {
            include $file_name;
        } else {
            $this->action_error("La vue n'existe pas !");
        }
        die();
    }

    protected function action_error($message = '')
    {
        $data = [
            'title' => "Error",
            'message' => $message,
        ];
        $this->render("message", $data);
    }

    public function debug($value,$die=true){
        echo "<pre>".print_r($value,true)."</pre>";
        if($die){
            die();
        }
    }

}
```

Code Controller_user.php :

```php
<?php

class Controller_user extends Controller {

    public function action_default() {
        $this->action_user();
    }

    public function action_user() {
        // Assurez-vous que l'utilisateur est connecté
        if (!isset($_SESSION['id_utilisateur'])) {
            header('Location: /stage_vehicule/index.php');
            exit();
        }

        $id_utilisateur = $_SESSION['id_utilisateur'];

        // Obtenir une instance du modèle
        $m = Model::getModel();

        // Récupérer les réservations de l'utilisateur
        $reservations = $m->getReservationsByUserId($id_utilisateur);
    }

}
```

Code Controller_admin.php :

```php
<?php

class Controller_admin extends Controller {

    public function action_default() {
        $this->action_voir_demandes();
    }

    public function action_voir_demandes() {
        // Assurez-vous que l'utilisateur est connecté et est un administrateur
        if (!isset($_SESSION['id_utilisateur']) || $_SESSION['roles'] != 'administrateur') {
            // Si l'utilisateur n'est pas connecté ou n'est pas un admin, redirigez-le vers la page de connexion
            header('Location: /stage_vehicule/index.php');
            exit();
        }

        $m = Model::getModel();

        // Récupérer toutes les demandes de réservation
        $demandes = $m->getAllDemandes();

        // Passer les données des demandes à la vue admin_voir_demandes
        $this->render('admin_voir_demandes', ['demandes' => $demandes]);
    }

    // Les méthodes pour approuver et refuser les demandes pourraient ressembler à ceci
    public function action_approuver_demande($id_demande) {
        $m = Model::getModel();
        $m->changerStatutDemande($id_demande, 'confirmée');
        header('Location: /stage_vehicule/controller_admin.php?action=voir_demandes');
    }

    public function action_refuser_demande($id_demande) {
        $m = Model::getModel();
        $m->changerStatutDemande($id_demande, 'refusée');
        header('Location: /stage_vehicule/controller_admin.php?action=voir_demandes');
    }

    // Ajoutez ici d'autres méthodes nécessaires pour gérer les actions administratives
}
```

Code Controller_login.php :

```php
<?php

class Controller_login extends Controller {

    public function action_login() {
        $errorLogin = "";
        $successLogin = "";

        if (isset($_POST["username"]) && isset($_POST["password"])) {
            $username = htmlspecialchars($_POST["username"]);
            $password = htmlspecialchars($_POST["password"]);

            $m = Model::getModel();
            $user = $m->findUserByUsername($username);

            if ($user !== false && password_verify($password, $user['mot_de_passe'])) {
                // Successful login
                $_SESSION['login'] = 1;
                $_SESSION['id_utilisateur'] = $user['id_utilisateur'];
                $_SESSION['nom_utilisateur'] = $user['nom_utilisateur'];
                $_SESSION['roles'] = $user['roles']; // Assuming roles are directly stored in $user, adjust if needed

                // Redirect based on role
                if ($user['roles'] == 'administrateur') {
                    header('Location: ?controller=voir_demandes'); // Mettre le chemin de la page d'accueil de l'administrateur
                } elseif ($user['roles'] == 'utilisateur') {
                    header('Location: ?controller=mes_reservations');
                }
                exit();
            } else {
                // Failed login
                $errorLogin = '<div style="color:red; text-align:center;">Le nom d\'utilisateur ou le mot de passe est incorrect.</div>';
            }
        }

        $data = ['errorLogin' => $errorLogin, 'successLogin' => $successLogin];
        $this->render('login', $data);
    }



        public function action_default() {
            $this->action_login();
        }
}
```

Code Controller_register.php :

```php
<?php

class Controller_register extends Controller {
    public function action_signup() {
        $errorUser = $errorMdp = $errorCreation = $successMsg = '';
        $id_utilisateur = $_SESSION['id_utilisateur'];
        if (!empty($_POST['username']) && !empty($_POST['password']) && !empty($_POST['password_c']) && !empty($_POST['role'])) {
            $username = htmlspecialchars($_POST['username']);
            $password = htmlspecialchars($_POST['password']);
            $password_c = htmlspecialchars($_POST['password_c']);
            $role = htmlspecialchars($_POST['role']);

            $m = Model::getModel();

            if ($m->usernameExists($username)) {
                $errorUser = '<p style="color:red; text-align:center;">Le nom d\'utilisateur existe déjà.</p>';
            } elseif ($password != $password_c) {
                $errorMdp = '<p style="color:red; text-align:center;">Les mots de passe ne correspondent pas.</p>';
            } else {
                $userId = $m->createUser($username, $password, $role);
                if ($userId) {
                    $successMsg = 'Le compte a été créé avec succès.';
                } else {
                    $errorCreation = 'Une erreur est survenue lors de la création du compte.';
                }
            }
        }

        $data = [
            'successMsg' => $successMsg,
            'errorUser'  => $errorUser,
            'errorMdp'   => $errorMdp,
            'errorCreation' => $errorCreation
        ];
        $this->render('register', $data);
    }

    public function action_default() {
        $this->action_signup();
    }
}

?>
```

Code Controller_support.php :

```php
<?php

class Controller_support extends Controller {

    public function action_default() {
        $this->action_support();
    }

    public function action_support() {
        // Assurez-vous que l'utilisateur est connecté
        if (!isset($_SESSION['id_utilisateur'])) {
            // Si l'utilisateur n'est pas connecté, redirigez-le vers la page de connexion
            header('Location: /stage_vehicule/index.php');
            exit();
        }

        // Affichez la page de support
        $this->render('support');
    }
}
```

Code Controller_mes_reservations.php :

```php
<?php

class Controller_mes_reservations extends Controller {

    public function action_voir_reservations() {
        $m = Model::getModel();
        $id_utilisateur = $_SESSION['id_utilisateur']; // Ensure the user is logged in and the session is started
        $reservations = $m->getReservationsByUserId($id_utilisateur);
        $this->render('mes_reservations', ['reservations' => $reservations]);
}


    public function action_default() {
        $this->action_voir_reservations();
    }

}
```

Code Controller_voir_demandes.php :

```php
<?php

class Controller_voir_demandes extends Controller {

    public function action_voir_demandes() {
        // Ajouté la vérification de l'administrateur ici
        if (!isset($_SESSION['roles']) || $_SESSION['roles'] != 'administrateur') {
            header('Location: /stage_vehicule/index.php');
            exit();
        }

        $m = Model::getModel();

        $demandes = $m->getAllDemandes();
        $this->render('voir_demandes', ['demandes' => $demandes]);
    }

    public function action_approuver_demande() {
        $id_demande = $_POST['id_demande'] ?? null;
        $modele_voiture = $_POST['modele_voiture'] ?? null;
        $numero_voiture = $_POST['numero_voiture'] ?? null;
        if ($id_demande && $modele_voiture && $numero_voiture) {
            $m = Model::getModel();
            $m->changerStatutDemande($id_demande, 'confirmée');
            $m->attribuerVoiture($id_demande, $modele_voiture, $numero_voiture);
            header('Location: ?controller=voir_demandes&action=voir_demandes');
        }
    }


    public function action_refuser_demande() {
        $id_demande = $_POST['id_demande'] ?? null;
        if ($id_demande) {
            $m = Model::getModel();
            if ($m->changerStatutDemande($id_demande, 'refusée')) {
                header("Location: ?controller=voir_demandes&action=refuser_demande");
            } else {
                echo "Erreur lors de la mise à jour du statut de la demande.";
            }
        } else {
            header('Location: ?controller=voir_demandes');
        }
    }


    // Assurez-vous d'avoir la méthode action_default
    public function action_default() {
        $this->action_voir_demandes();
    }

    // Vous pouvez ajouter ici d'autres méthodes nécessaires
}
```

Code Controller_demander_vehicule.php :

```php
<?php

class Controller_demander_vehicule extends Controller {

    private function validateAndSanitize($data) {
        $sanitizedData = [];
        $isHebdomadaire = isset($data['type_demande']) && $data['type_demande'] === 'hebdomadaire';

        // Fields common to both types of requests
        $commonFields = ['type_demande', 'nomDemandeur'];

        // Fields specific to 'ponctuelle' requests
        $ponctuelleFields = [
            'serviceDemandeur', 'beneficiaire', 'destination', 'nomTransportes',
            'visaChefService', 'noOrdrePlanning', 'signatureUtilisateur',
            'periodeUtilisation', 'periodeUtilisationDe', 'periodeUtilisationA'
        ];

        // Fields specific to 'hebdomadaire' requests
        $hebdomadaireFields = [
            'semaineNo', 'beneficiaireHebdo', 'dateCreationHebdo', 'periodeHebdo',
            'periodeHebdoA', 'nomDemandeurHebdo', 'destinationHebdo', 'lundi', 'destination_lundi',
            'mardi', 'destination_mardi', 'mercredi', 'destination_mercredi',
            'jeudi', 'destination_jeudi', 'vendredi', 'destination_vendredi',
            'samedi', 'destination_samedi', 'dimanche', 'destination_dimanche'];

        // Determine which set of fields to use
        $fieldsToUse = $isHebdomadaire ? array_merge($commonFields, $hebdomadaireFields) : array_merge($commonFields, $ponctuelleFields);

        // Validate and sanitize the fields
        foreach ($fieldsToUse as $field) {
            if (isset($data[$field])) {
                $sanitizedData[$field] = filter_var($data[$field], FILTER_SANITIZE_FULL_SPECIAL_CHARS);
            }
        }

        // Additional validation for fields like dates and booleans
        if (!$isHebdomadaire && isset($data['periodeUtilisation'])) {
            // Validate 'periodeUtilisation' only for ponctuelle requests
            $sanitizedData['periodeUtilisation'] = $this->validateDate($data['periodeUtilisation']) ? $data['periodeUtilisation'] : null;
        }

        if (isset($data['chauffeur'])) {
            $sanitizedData['chauffeur'] = filter_var($data['chauffeur'], FILTER_VALIDATE_BOOLEAN);
        }

        return $sanitizedData;
    }

    private function validateDate($date, $format = 'Y-m-d') {
        $d = DateTime::createFromFormat($format, $date);
        return $d && $d->format($format) === $date;
    }

    public function action_demander_vehicule() {
        // Vérifie si la requête est une soumission POST
        if ($_SERVER['REQUEST_METHOD'] === 'POST') {
            $formData = $this->validateAndSanitize($_POST);
            if (!empty($formData)) {
                $m = Model::getModel();
                $result = $m->createReservation($formData, $_SESSION['id_utilisateur']);
                if ($result) {
                    echo "<script>alert('Demande soumise avec succès.');</script>";
                } else {
                    echo "<script>alert('Erreur lors de la soumission de la demande.');</script>";
                }

            } else {
                echo "<script>alert('Données du formulaire invalides.');</script>";
            }
        }
        $this->render('demander_vehicule');
    }
```

Code script.js :

```javascript
// JavaScript pour gérer l'ouverture et la fermeture des modales
document.getElementById('demandePonctuelleBtn').onclick = function() {
    document.getElementById('demandePonctuelleModal').style.display = 'block';
  }

  document.getElementById('demandeHebdomadaireBtn').onclick = function() {
    document.getElementById('demandeHebdomadaireModal').style.display = 'block';
  }

  // Fermer la modale si l'utilisateur clique en dehors
  window.onclick = function(event) {
    if (event.target.className === 'modal') {
      event.target.style.display = 'none';
    }
  }

  // Ajouter un événement onclick pour le bouton de fermeture
var closeBtns = document.getElementsByClassName("close");
for (var i = 0; i < closeBtns.length; i++) {
  closeBtns[i].onclick = function() {
    var modal = this.parentElement.parentElement;
    modal.style.display = "none";
  }
}
```

Code style.css :

```css
/* Home */
* {
    margin: 0;
    padding: 0;
    box-sizing: border-box;

}

body, html {
    height: 100%;
    font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;

}

.view-home {
    display: flex;
    height: 100%;

}

.sidebar {
    background-color: #034078; /* Adjusted to match the screenshot */
    color: white;
    width: 250px; /* Adjust width as needed */
    padding: 20px;
    display: flex;
    flex-direction: column;
    position : fixed;
    height: 100%;
    z-index: 1;
}

.logo img {
    width: 150px;
    height: auto;
    display: block;
    margin-bottom: 2rem;
    margin-left: auto;
    margin-right: auto;
}

.nav-menu {
    display: flex;
    flex-direction: column;
}

.nav-item {
    padding: 15px 20px;
    color: white;
    text-decoration: none;
    font-size: 1rem;
    margin-bottom: 10px;
    transition: background-color 0.3s, padding-left 0.3s;
}

.nav-item:hover {
    background-color: #026044; /* Slightly lighter blue for hover state */
    padding-left: 30px; /* Indent text to the right on hover */
}

.main-content {

    flex-grow: 1;
    background-color: #EEEEEE; /* Right-side background */
}

.top-header {
    background-color: #025588;
    color: white;
    padding: 1rem;
    text-align: right;
    position: fixed;
    width: 100%;
```